india-tax-engine: Product Brief and Build Decision

Project proposal for an npm package · November 2026 · Status: proposed, ready to build

1. One-line summary

`india-tax-engine` is a small, dependency-free TypeScript library that calculates Indian income tax for individuals and returns the answer together with a step-by-step explanation of how it was reached. Tax rules are stored as versioned data per tax year, so a new Budget means adding a file, not rewriting code.

2. The problem

A developer who needs to answer “how much tax will this person pay?” today has three poor options:

1. **Copy a formula from a blog.** Blogs disagree and go out of date.
2. **Use a website calculator.** It cannot be embedded in your own app or tested automatically.
3. **Write the logic from scratch.** This means learning slabs, the section 87A rebate, marginal relief, surcharge, cess and two tax laws.

The rules also changed recently. The Income-tax Act, 2025 replaces the 1961 Act from 1 April 2026. Financial year 2025-26 is still governed by the old Act, while 2026-27 onwards follows the new one, and the terms “financial year” and “assessment year” are replaced by “tax year”. Code with hardcoded numbers breaks every time the law or a Budget changes.

The example that shows why it matters

Under the new regime, income up to ₹12,00,000 pays zero tax because the section 87A rebate cancels the slab tax of ₹60,000. A naive program at ₹12,10,000 charges the full slab tax of about ₹61,500, a jump of ₹61,500 for ₹10,000 of extra income. The law prevents this with **marginal relief**, which caps the tax at the income earned above ₹12,00,000. The correct tax at ₹12,10,000 is about ₹10,000 plus 4% cess.

Many simple calculators miss this rule, and it matters exactly where many salaried people earn.

3. Target users

User	What they need	Why this library fits
Student and indie developers building salary or finance apps	Correct tax logic without learning tax law first	One function call, readable output
Small HR and payroll tools	A transparent estimate employees can understand	Step-by-step explanation
Fintech and policy-tech prototypes	Rules that can be updated yearly	Rules-as-data, versioned per tax year
AI apps and agents	A trustworthy calculation tool instead of a language model guessing arithmetic	Deterministic, testable output
Our own PolicyLens AI project	An eligibility and tax-impact component	Direct reuse, real usage

Not for: chartered accountants, official return filing, or anyone needing legally binding advice.

4. How the product flows

```
Developer's app
      |  calculateTax({ taxYear, regime, income, deductions })
      v
1.  Validate input (numbers, known tax year, known regime)
      v
2.  Convert all money to whole paise (no floating-point errors)
      v
3.  Load the rules file for that tax year (JSON)
      v
4.  Compute taxable income (income minus allowed deductions)
      v
5.  Apply slab rates, layer by layer
      v
6.  Apply section 87A rebate if eligible
      v
7.  Apply marginal relief if income is just above the rebate limit
      v
8.  Apply surcharge (high incomes, added last if time allows)
      v
9.  Add 4% health and education cess
      v
10. Return { totalTax, steps[], regime, taxYear, warnings[] }
```

Comparison mode: `compareRegimes(input)` runs the same flow for the old and new regime and reports which is cheaper and by how much.

Example output

```
Slab tax on ₹12,10,000 ........ ₹61,500
Rebate 87A ..................... not available (income above ₹12L)
Marginal relief ............... -₹51,500
Tax after relief .............. ₹10,000
Cess (4%) ...................... ₹400
Total tax ...................... ₹10,400
```

5. Is it still relevant?

Yes, and the timing helps. Three facts support this, drawn from our research in November 2026:

- The Income-tax Act, 2025 took effect on 1 April 2026, so calculators and apps built on 2025 logic need review.
- The new regime is now the default, which makes regime-comparison questions common.
- Sources report that slab rates themselves did not change for tax year 2026-27, so the structural change (law, terminology, year handling) is where tools break, not the numbers.

Caveat on demand. Relevance is not the same as downloads. The problem is real and recurring, but we could not measure how many developers search for this on npm, because npm download statistics were not accessible during research.

6. What already exists (competition check)

Researched on the npm registry in November 2026. Registry search is fuzzy, so this list may be incomplete and must be re-checked before publishing.

Package	What it does	Gap relative to our idea
`itr-agent`	Local MCP server for Indian income tax, compares regimes	Built for AI agents, not a plain library

Package	What it does	Gap relative to our idea
`calcsuite-react`	Decimal-exact calculators for India and US in React	UI-focused, not a rules engine
`@mudraforge/statutory`	Open knowledge format and agent skill for Indian statutory data	Different goal, not a calculator library
GST packages (gstin-utils, @business-swift/gst-engine, hsn-code-package)	GST validation and calculation	Different tax, crowded area, we avoid it
Other countries (UK, Dutch, Nigerian, Bangladeshi tax engines)	Country-specific income tax	Proves the pattern has demand

We found no plain, dependency-free TypeScript library for Indian income tax that combines all four of our differentiators below. We did not and cannot prove none exists.

7. What we add that is new

1. **It explains itself.** Every result includes readable steps, so users and developers can trust and debug it.
2. **Rules are data, not code.** Each tax year is a JSON file. A Budget change means a new file plus tests.
3. **Exact money maths.** All amounts are integer paise, so there is no rounding drift.
4. **Handles the two-law transition.** Tax year 2025-26 and 2026-27 are modelled separately.
5. **Regime comparison in one call.**

8. Scope of version 1 (to fit about 12 to 13 hours)

Included

- Resident individuals, salaried
- New and old regime
- Tax years 2025-26 and 2026-27
- Slabs, standard deduction, section 87A rebate, marginal relief, 4% cess
- Step-by-step explanation and regime comparison
- Surcharge, only if time remains

Excluded for now

- Capital gains, HRA, crypto, business income, NRIs, senior-citizen slabs

9. Build roadmap (about 12 hours)

Block	Hours
Learn the rules, collect official numbers	1.5
Rules JSON format and loader	1.5
Money helpers (paise) and input validation	1.0
Slab calculation	1.5
Rebate and marginal relief	2.0
Cess, explanation steps, regime comparison	1.5
Tests with hand-verified cases	1.5
README, packaging, publish	1.5

10. Risks and how we handle them

Risk	Mitigation
Wrong tax numbers damage trust	Verify every figure against the official incometax.gov.in and Finance Act text; cite the source in each rules file
Blogs conflict on details	Treat blogs as leads only, never as the source of truth
Someone treats output as tax advice	Clear README disclaimer: informational estimate, not tax advice
Low downloads	Do not rely on downloads for success; see section 12
A similar package already exists	Re-check npm and GitHub before publishing and narrow the angle if needed
Scope creep past 13 hours	Stick to section 8; log extra ideas as future work

11. Is it worth building?

For learning and CV: yes. It teaches rules-as-data design, versioning, exact money arithmetic, boundary testing and TypeScript API design. These skills apply to payroll, banking, insurance and government technology, and it connects to your PolicyLens AI project.

For downloads: uncertain. The need is real, but a new package from an unknown author typically starts with few real users. Treat downloads as a bonus.

Decision rule. Build it if the answers to these three are yes:

1. Can we source official numbers for both tax years and both regimes?
2. Does a final npm and GitHub check confirm no near-identical library?
3. Do you want the skills in this section more than a guaranteed audience?

12. How to show it was worth it (for a teacher or reviewer)

- Published npm package with a live link and version
- Passing tests, with hand-checked cases such as ₹12,00,000 and ₹12,10,000
- A README with install, usage, API, a sources list and a disclaimer
- A GitHub repository with a clear commit history

- Real use inside PolicyLens AI
- Download statistics as supporting evidence only, never fabricated or inflated

13. Open items to confirm before coding

- Exact new-regime slab for ₹12 to ₹16 lakh and the standard deduction amounts for each year, from official sources
- Surcharge thresholds and the cap in the new regime
- Old-regime slabs and deduction limits for both tax years
- Final package name availability (`india-tax-engine` returned no match when checked, but re-check on publish day)

14. Research sources

- npm registry searches, November 2026 (competition check)
- Economic Times and Motilal Oswal on the new regime and slabs after Budget 2026
- Taxaj, India Briefing and TaxGuru on the Income-tax Act, 2025 and its timeline
- Tally Solutions and Go Digit on the section 87A rebate and marginal relief
- Official source to rely on for final numbers: incometax.gov.in